Note that `print` and `echo` have no parentheses `()`, because they are "language constructs," not functions. This distinction might be beyond you at the moment, but don't worry: Chapter 3, "Functions," contains everything you need to understand what PHP functions are and how they work. Skip ahead if you like!

The Output Buffer

I've mentioned output buffering twice so far in this section, but what is it? Well, it's quite simple: data output by your program can be sent immediately, or it can be accumulated in a temporary store for sending in one large chunk later on. This can be quite useful in the context of HTTP requests, where HTTP protocol header data has to be sent before the response body. A complex program might have already started to form the body before all the headers are calculated and sent. In particular, the `Content-Length:` header needs to know the full size (in bytes) of the response body, which you don't necessarily know until all your code is done. If your script has already started to spit out the response, then tries to send an HTTP header this would constitute a protocol error. PHP allows for "user-level" output buffers – that is, buffers that are under the control of your program. Here's a quick example:

```
ob_start();

print "What are we holding onto, Sam?";
print PHP_EOL;
print "That there's some good in this world, Mr. Frodo... and it's worth
fighting for.";

$buffered = ob_get_clean();
print "Buffer contents:" . PHP_EOL;
print $buffered;

Output:
Buffer contents:
What are we holding onto, Sam?
That there's some good in this world, Mr. Frodo... and it's worth
fighting for.
```